Pandas Guide

Pandas is a powerful library for data manipulation and analysis in Python. It provides data structures like Series and DataFrame, which allow you to work with structured data efficiently.

1. Installation & Basics

To install pandas via terminal, you can use:

```
pip install pandas
```

In [2]:
```
import pandas as pd
```

In [3]:
```
# Series
series = pd.Series([1, 2, 3, 4, 5])
print("Series:")
print(series)

# DataFrame
data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [30, 25, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
}
df = pd.DataFrame(data)
print("\nDataFrame:")
print(df)
```

```
Series:
0    1
1    2
2    3
3    4
4    5
dtype: int64

DataFrame:
      Name  Age         City
0    Alice   30     New York
1      Bob   25  Los Angeles
2  Charlie   35      Chicago
```

2. Creating a DataFrame from a CSV file

In [9]:
```
df_csv = pd.read_csv("./assets/data.csv")

# Exploring DataFrame (first 5 rows by default)
print("Exploring DataFrame (first 5 rows by default):")
print(df_csv.head())

# Exploring DataFrame (last 5 rows)
```

```
print("\nExploring DataFrame (last 5 rows):")
print(df_csv.tail())

# Exploring DataFrame (specific number of rows)
print("\nExploring DataFrame (specific number of rows):")
display(df_csv)

# Reading CSV with a different separator
df = pd.read_csv("./assets/data.csv", sep=",")
print("\nReading CSV with a different separator:")
print(df.head())

# Reading CSV without header
df = pd.read_csv("./assets/data.csv", header=None)
print("\nReading CSV without header:")
print(df.head())

# Reading CSV with custom column names
df = pd.read_csv("./assets/data.csv", names=["Column1", "Column2", "Colum
print("\nReading CSV with custom column names:")
print(df.head())

# Reading CSV and setting a column as index
df = pd.read_csv("./assets/data.csv", index_col="Name")
print("\nReading CSV and setting a column as index:")
print(df.head())

# Reading only specific columns from CSV
df = pd.read_csv("./assets/data.csv", usecols=["Name", "Age", "City"])
print("\nReading only specific columns from CSV:")
print(df.head())

# Skipping the first row of the CSV file
df = pd.read_csv("./assets/data.csv", skiprows=1)
print("\nSkipping the first row of the CSV file:")
print(df.head())

# Reading only the first 3 rows of the CSV file
df = pd.read_csv("./assets/data.csv", nrows=3)
print("\nReading only the first 3 rows of the CSV file:")
print(df.head())

# Treating specific values as NaN
df = pd.read_csv("./assets/data.csv", na_values=["NA", "N/A"])
print("\nTreating specific values as NaN:")
print(df.head())

# Parsing specific columns as dates
df = pd.read_csv("./assets/data.csv", parse_dates=["Date"])
print("\nParsing specific columns as dates:")
print(df.head())

# Specifying data types for columns
df = pd.read_csv("./assets/data.csv", dtype={"Name": str, "Age": int, "Ci
print("\nSpecifying data types for columns:")
print(df.head())

# Specifying encoding when reading CSV file
df = pd.read_csv("./assets/data.csv", encoding="utf-8")
```

```
print("\nSpecifying encoding when reading CSV file:")
print(df.head())
```

```
Exploring DataFrame (first 5 rows by default):
      Name  Age           City        Date
0    Alice   30       New York  2023-01-15
1      Bob   25    Los Angeles  2023-02-20
2  Charlie   35        Chicago  2023-03-10
3    David   28  San Francisco  2023-04-05
4      Eva   32       New York  2023-05-12

Exploring DataFrame (last 5 rows):
      Name  Age           City        Date
2  Charlie   35        Chicago  2023-03-10
3    David   28  San Francisco  2023-04-05
4      Eva   32       New York  2023-05-12
5    Frank   27    Los Angeles  2023-06-18
6    Grace   29        Chicago  2023-07-22

Exploring DataFrame (specific number of rows):
```

	Name	Age	City	Date
0	Alice	30	New York	2023-01-15
1	Bob	25	Los Angeles	2023-02-20
2	Charlie	35	Chicago	2023-03-10
3	David	28	San Francisco	2023-04-05
4	Eva	32	New York	2023-05-12
5	Frank	27	Los Angeles	2023-06-18
6	Grace	29	Chicago	2023-07-22

```
Reading CSV with a different separator:
      Name  Age           City        Date
0    Alice   30       New York  2023-01-15
1      Bob   25    Los Angeles  2023-02-20
2  Charlie   35        Chicago  2023-03-10
3    David   28  San Francisco  2023-04-05
4      Eva   32       New York  2023-05-12

Reading CSV without header:
         0    1              2           3
0     Name  Age           City        Date
1    Alice   30       New York  2023-01-15
2      Bob   25    Los Angeles  2023-02-20
3  Charlie   35        Chicago  2023-03-10
4    David   28  San Francisco  2023-04-05

Reading CSV with custom column names:
        Column1        Column2     Column3
Name        Age           City        Date
Alice        30       New York  2023-01-15
Bob          25    Los Angeles  2023-02-20
Charlie      35        Chicago  2023-03-10
David        28  San Francisco  2023-04-05

Reading CSV and setting a column as index:
         Age           City        Date
Name
Alice     30       New York  2023-01-15
Bob       25    Los Angeles  2023-02-20
Charlie   35        Chicago  2023-03-10
David     28  San Francisco  2023-04-05
Eva       32       New York  2023-05-12

Reading only specific columns from CSV:
      Name  Age           City
0    Alice   30       New York
1      Bob   25    Los Angeles
2  Charlie   35        Chicago
3    David   28  San Francisco
4      Eva   32       New York

Skipping the first row of the CSV file:
     Alice  30       New York  2023-01-15
0      Bob  25    Los Angeles  2023-02-20
1  Charlie  35        Chicago  2023-03-10
2    David  28  San Francisco  2023-04-05
3      Eva  32       New York  2023-05-12
4    Frank  27    Los Angeles  2023-06-18

Reading only the first 3 rows of the CSV file:
      Name  Age         City        Date
0    Alice   30     New York  2023-01-15
1      Bob   25  Los Angeles  2023-02-20
2  Charlie   35      Chicago  2023-03-10

Treating specific values as NaN:
      Name  Age           City        Date
0    Alice   30       New York  2023-01-15
1      Bob   25    Los Angeles  2023-02-20
2  Charlie   35        Chicago  2023-03-10
```

```
3    David   28  San Francisco  2023-04-05
4      Eva   32       New York  2023-05-12

Parsing specific columns as dates:
      Name  Age           City       Date
0    Alice   30       New York 2023-01-15
1      Bob   25    Los Angeles 2023-02-20
2  Charlie   35        Chicago 2023-03-10
3    David   28  San Francisco 2023-04-05
4      Eva   32       New York 2023-05-12

Specifying data types for columns:
      Name  Age           City        Date
0    Alice   30       New York  2023-01-15
1      Bob   25    Los Angeles  2023-02-20
2  Charlie   35        Chicago  2023-03-10
3    David   28  San Francisco  2023-04-05
4      Eva   32       New York  2023-05-12

Specifying encoding when reading CSV file:
      Name  Age           City        Date
0    Alice   30       New York  2023-01-15
1      Bob   25    Los Angeles  2023-02-20
2  Charlie   35        Chicago  2023-03-10
3    David   28  San Francisco  2023-04-05
4      Eva   32       New York  2023-05-12
```

3. Exploring the DataFrame

In [5]:
```python
# DataFrame Information
print("DataFrame Information:")
print("Info:", df.info())

# Statistical Summary
print("\nStatistical Summary:")
print("Describe:", df.describe())

# DataFrame Shape
print("\nDataFrame Shape:")
print("Shape:", df.shape)
```

```
DataFrame Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype 
---  ------  --------------  ----- 
 0   Name    5 non-null      object
 1   Age     5 non-null      int64 
 2   City    5 non-null      object
 3   Date    5 non-null      object
dtypes: int64(1), object(3)
memory usage: 292.0+ bytes
Info: None

Statistical Summary:
Describe:              Age
count   5.000000
mean   30.000000
std     3.807887
min    25.000000
25%    28.000000
50%    30.000000
75%    32.000000
max    35.000000

DataFrame Shape:
Shape: (5, 4)
```

4. Accessing and Filtering Data

In [6]:
```python
# Accessing data (Series)
print("Accessing data (Series):")

print("Age column:")
print(df["Age"])
print("\nName and City columns:")
print(df[["Name", "City"]])

# Accessing data (DataFrame)
print("\nAccessing data (DataFrame):")

print("First row:")
print(df.iloc[0])
print("\nFirst two rows:")
print(df.iloc[0:2])

# Accessing data by label
print("\nAccessing data by label:")
print("Name in first row:")
print(df.loc[0, "Name"])

# Sorting data (DataFrame)
print("\nSorting data (DataFrame):")
print("Sorted by Age:")
print(df.sort_values(by="Age"))

# Filtering data (DataFrame)
print("\nFiltering data (DataFrame):")
```

```
print("Age > 30:")
print(df[df["Age"] > 30])
print("\nCity == New York:")
print(df[df["City"] == "New York"])
```

```
Accessing data (Series):
Age column:
0    30
1    25
2    35
3    28
4    32
Name: Age, dtype: int64

Name and City columns:
      Name           City
0    Alice       New York
1      Bob    Los Angeles
2  Charlie        Chicago
3    David  San Francisco
4      Eva       New York

Accessing data (DataFrame):
First row:
Name         Alice
Age             30
City      New York
Date    2023-01-15
Name: 0, dtype: object

First two rows:
    Name  Age         City        Date
0  Alice   30     New York  2023-01-15
1    Bob   25  Los Angeles  2023-02-20

Accessing data by label:
Name in first row:
Alice

Sorting data (DataFrame):
Sorted by Age:
      Name  Age           City        Date
1      Bob   25    Los Angeles  2023-02-20
3    David   28  San Francisco  2023-04-05
0    Alice   30       New York  2023-01-15
4      Eva   32       New York  2023-05-12
2  Charlie   35        Chicago  2023-03-10

Filtering data (DataFrame):
Age > 30:
      Name  Age      City        Date
2  Charlie   35   Chicago  2023-03-10
4      Eva   32  New York  2023-05-12

City == New York:
    Name  Age      City        Date
0  Alice   30  New York  2023-01-15
4    Eva   32  New York  2023-05-12
```

5. Modifying and Saving Data

In [7]:
```python
# Modifying data (DataFrame)
print("\nModifying data (DataFrame):")

df["Age"] = df["Age"] + 1
print("Incremented Age:")
print(df)

df.drop(columns=["City"], inplace=True)
print("\nDropped City column:")
print(df)

df.loc[df["Name"] == "Alice", "Age"] = 32
print("\nUpdated Alice's Age:")
print(df)

df.drop(index=0, inplace=True)
print("\nDropped first row:")
print(df)

# Saving data
print("\nSaving data:")
df.to_csv("./assets/updated_data.csv", index=False)
print("\nDataFrame successfully saved to 'updated_data.csv'")
```

```
Modifying data (DataFrame):
Incremented Age:
      Name  Age           City        Date
0    Alice   31       New York  2023-01-15
1      Bob   26    Los Angeles  2023-02-20
2  Charlie   36        Chicago  2023-03-10
3    David   29  San Francisco  2023-04-05
4      Eva   33       New York  2023-05-12

Dropped City column:
      Name  Age        Date
0    Alice   31  2023-01-15
1      Bob   26  2023-02-20
2  Charlie   36  2023-03-10
3    David   29  2023-04-05
4      Eva   33  2023-05-12

Updated Alice's Age:
      Name  Age        Date
0    Alice   32  2023-01-15
1      Bob   26  2023-02-20
2  Charlie   36  2023-03-10
3    David   29  2023-04-05
4      Eva   33  2023-05-12

Dropped first row:
      Name  Age        Date
1      Bob   26  2023-02-20
2  Charlie   36  2023-03-10
3    David   29  2023-04-05
4      Eva   33  2023-05-12

Saving data:

DataFrame successfully saved to 'updated_data.csv'
```